Network Programming (IE2102)
2nd Year, 2nd Semester

Assignment

Design and Implementation of a Secure Multiprocessor TCP Application Server with Custom Protocol and Session Management

Submitted to

Sri Lanka Institute of Information Technology

IT24100139

Pinthu D.I.U.

In partial fulfillment of the requirements for the

Bachelor of Science Special Honors Degree in Information Technology

5 April 2026

Declaration

I certify that this report does not incorporate without acknowledgement, any material previously submitted for a degree or diploma in any university, and to the best of my knowledge and belief, it does not contain any material previously published or written by another person, except where due reference is made in text

Table of Contents

1. System Design and Architecture Overview 1
2. My Environment Configurations 3
3. Server Execution and Port Binding 5
4. Protocol and Authentication 5
5. Multiprocessing and Concurrency 7
6. Persistent Audit Logging 8

1. System Design and Architecture Overview

The system is implemented as a secure multiprocessor TCP application server that incorporates a custom protocol and robust session management. It adopts a classic client-server architecture and utilizes standard socket communication over TCP.

1.1 Multiprocessing Architecture

- **Process Management:** The server employs a multiprocess architecture, implemented using the fork() system call.
- **Connection Handling:** The parent is exclusively responsible for accepting new client connections. Upon successful connection, it creates a child process to manage the entire session for that client.
- **Process Cleanup:** To ensure system stability and prevent resource leaks, the server implements SIGCHLD signal handling with WAITPID () to clean up terminated child processes and prevent zombie processes.

1.2 Custom TCP Protocol (Explicit Framing)

- **Message Framing:** To ensure reliable data transmission over a TCP stream, the system employs an explicit framing architecture.
- **Header Format:** Each client message begins with a strict header format: LEN:<n>\n, where <n> specifies the exact byte count of the payload.
- **Buffer Management:** The server is designed to safely process partial recv() calls, handle multiple messages with a single buffer, and reject invalid lengths or payloads exceeding 4096 bytes.
- **Standardized Responses:** Server responses adhere to a consistent format that includes status codes and the server ID, such as OK <code> SID: <SID> <message> or ERR<code> SID: <SID> <message>.

1.3 Security and Session Management

- **Credential Storage:** User passwords are never stored in plain text. The system uses salted hashing to ensure secure credential storage.
- **Token Authentication:** After successful login, the server generates and issues a unique session token to the client.
- **Access Control:** Access to protected commands requires the client to present an active session to the client.

1.4 Abuse Protection and Auditing

- **Traffic Control:** The server implements per-client rate limiting to prevent span and denial-of-service attacks.
- **Brute-Force Prevention:** The system employs a login lockout mechanism to protect accounts from brute-force attacks.
- **Validation:** Strict username validation and payload overflow rejections are enforced at the protocol level.
- **Persistent Logging:** All significant events are recorded in a persistent audit log. This log tracks the timestamp, client IP and port, child process ID, username and command result for system monitoring.

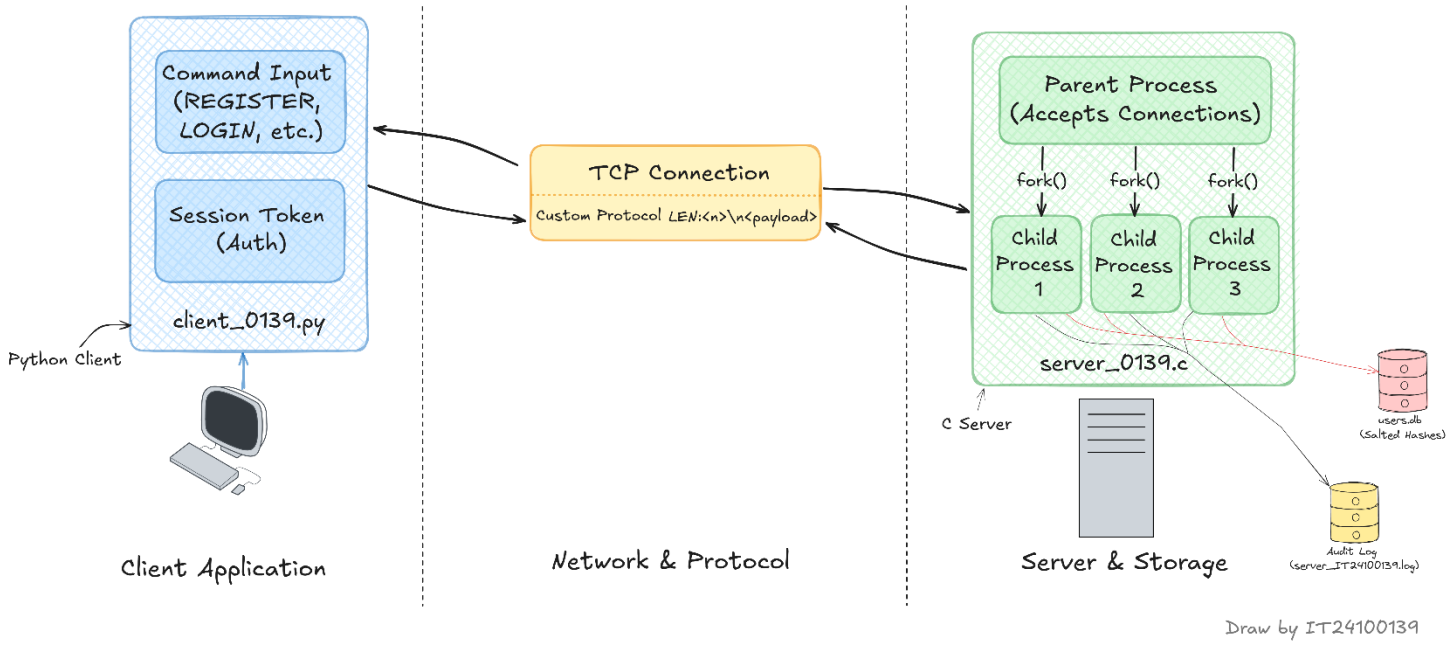

Figure 1.1: Architecture Diagram

2. My Environment Configurations

This section describes how the development and execution environments were configured for this assignment, as specified. The implementation follows the required file naming conventions, port configurations, server identification format, logging format, and storage directory structure based on my registration number, **IT24100139.**

2.1 File Naming Configuration

According to the assignment requirements, file names must be derived from the registration number. For my registration number 'IT24100139', the configured file names are:

- **Server source file:** *server_0139.c*
- **Client source file:** *client_0139.py*
- **Makefile:** *Makefile_0139*

2.2 Server Port Configuration

- The assignment requirement: PORT = 50 + last 3 digits of registration number.
- My configured server port: ***50139.***

2.3 Server ID (SID) Configuration

- The assignment requirement: SID = <middle4digits>'
- My configured server ID**: 1001**.

2.4 Log File Configuration

- The assignment requirement: server_<regno>.log
- My configured log file: **server_IT24100139.log**

2.5 Data Storage Directory Structure

- The assignment requirement: /srv/ie2102/<regno>/<username>/
- My implementation: **/srv/ie2102/IT24100139/<username>/**

2.6 Host Name Configuration

```
isuru@IT24100139:~/Desktop/NP$ hostname
IT24100139
isuru@IT24100139:~/Desktop/NP$ hostnamectl
 Static hostname: IT24100139
       Icon name: computer-vm
         Chassis: vm
      Machine ID: a01b9ab31dd84912928af91e2aa13b7c
         Boot ID: fc5c976a29d642d8a8a6fb01f653a0fc
  Virtualization: oracle
Operating System: Ubuntu 24.04.4 LTS
          Kernel: Linux 6.17.0-20-generic
    Architecture: x86-64
 Hardware Vendor: innotek GmbH
  Hardware Model: VirtualBox
Firmware Version: VirtualBox
   Firmware Date: Fri 2006-12-01
    Firmware Age: 19y 4month 6d
isuru@IT24100139:~/Desktop/NP$
```

Figure 2.1: Hostname Configuration of the server environment

This screenshot verifies the hostname configuration of the ubuntu environment using the hostname and hostnamectl commands. The system's static hostname has been correctly configured to match the student registration number (IT24100139)

2.7 Network Configuration

```
isuru@IT24100139: ~/Desktop/NP    isuru@IT24100139: ~/Desktop/NP    isuru@IT24100139: ~/Desktop/NP
isuru@IT24100139:~/Desktop/NP$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:aa:e1:08 brd ff:ff:ff:ff:ff:ff
    inet 192.168.8.186/24 brd 192.168.8.255 scope global dynamic noprefixroute enp0s3
       valid_lft 64242sec preferred_lft 64242sec
    inet6 2402:4000:2351:3b51:cf5f:45b6:d570:a2ab/64 scope global temporary dynamic
       valid_lft 291sec preferred_lft 111sec
    inet6 2402:4000:2351:3b51:a00:27ff:feaa:e108/64 scope global dynamic mngtmpaddr
       valid_lft 291sec preferred_lft 111sec
    inet6 fe80::a00:27ff:feaa:e108/64 scope link
       valid_lft forever preferred_lft forever
isuru@IT24100139:~/Desktop/NP$ echo "IT24100139"
IT24100139
```

Figure 2.2: Network Configuration of the server environment

This Screenshot demonstrates the network configuration of the server machine using the '**ip a**' command. The server's primary network interfaced (enp0s3) is assigned the IP address 192.168.8.186, which will be used by the python client to establish the TCP connection.

3. Server Execution and Port Binding

3.1 Building the server

```
isuru@IT24100139: ~/Desktop/NP        isuru@IT24100139: ~/Desktop/NP
isuru@IT24100139:~/Desktop/NP$ make -f Makefile_0139
gcc -Wall -Wextra -O2 -o server_0139 server_0139.c -lssl -lcrypto -lpthread
isuru@IT24100139:~/Desktop/NP$ echo "IT24100139"
IT24100139
isuru@IT24100139:~/Desktop/NP$
```

Figure 3.1: Building the server using the Makefile

This Screenshot demonstrates the successful compilation of the C server application using the **make** utility. The command **make -f Makefile_0139** was executed to build the executable from the source code.

3.2 Server Verification

```
isuru@IT24100139:~/Desktop/NP$ echo "IT24100139"
IT24100139
isuru@IT24100139:~/Desktop/NP$ ./server_0139
[IE2102] SID:1001  Port:50139  RegNo:IT24100139  Log:/srv/ie2102/IT24100139/server_IT24100139.log
```

Figure 3.2: Server Verification

```
isuru@IT24100139:~/Desktop/NP$ ss -tuln | grep 50139
tcp   LISTEN 0      128          0.0.0.0:50139      0.0.0.0:*
isuru@IT24100139:~/Desktop/NP$ echo "IT24100139"
IT24100139
```

Figure 3.3: Port binding

Those screenshots verify that the TCP server is currently running and successfully bound to the correct assigned port. The **SS** command output confirms that the server is actively listening for incoming client connections on port 50139.

4. Protocol and Authentication

4.1 Custom TCP Framing

```
isuru@IT24100139: ~/Desktop/NP   isuru@IT24100139: ~/Desktop/NP   isuru@IT24100139: ~/Desktop/NP
isuru@IT24100139:~/Desktop/NP$ python3 client_0139.py
[IE2102 Client] Connecting to 127.0.0.1:50139 ...
[SERVER] OK 100 SID:1001 Welcome to IE2102 Server [IT24100139]. Commands: REGISTER LOGIN TOKEN LOGOUT WHOAMI ECHO TIME QUIT
>> LOGIN isuru 12345678
[SERVER] OK 200 SID:1001 Login successful. TOKEN:ec7eb211f85111efb6de84cc98c89c0c697922da0ff92a7045c82eb112dfc4ad  (expires after 300 sec inactivity)
[HINT]   To use protected commands, send: TOKEN ec7eb211f85111efb6de84cc98c89c0c697922da0ff92a7045c82eb112dfc4ad
>> TOKEN ec7eb211f85111efb6de84cc98c89c0c697922da0ff92a7045c82eb112dfc4ad
[SERVER] OK 200 SID:1001 Token accepted. Session active.
>> ECHO Hello
[SERVER] OK 200 SID:1001 ECHO: Hello
>>
```

Figure 4.1: Test Custom TCP framing with ECHO hello

This screenshot demonstrates the custom framing architecture in action. The Python client automatically prepends the exact payload length(**LEN: <n>**) to the **ECHO hello** message behind the scenes. The C server successfully parses this framed message and returns the exact payload, correctly including the required server ID (**SID: 1001**) in its response header.

4.2 User Registration

```
isuru@IT24100139:~/Desktop/NP$ python3 client_0139.py
[IE2102 Client] Connecting to 127.0.0.1:50139 ...
[SERVER] OK 100 SID:1001 Welcome to IE2102 Server [IT241
>> LOGIN isuru 12345678
[SERVER] OK 200 SID:1001 Login successful. TOKEN:ec7eb21
[HINT]   To use protected commands, send: TOKEN ec7eb211
>> TOKEN ec7eb211f85111efb6de84cc98c89c0c697922da0ff92a7
[SERVER] OK 200 SID:1001 Token accepted. Session active.
>> ECHO Hello
[SERVER] OK 200 SID:1001 ECHO: Hello
>> REGISTER Rasiru qwer1234
[SERVER] OK 200 SID:1001 Registration successful.
```

Figure 4.2: Successful user registration

This image shows the client successfully executing the **REGISTER** command. The server validates the request, securely processes the credentials using salted hashing, and returns a success response confirming the account creation.

4.3 User Login and Session Token

```
>> LOGIN Rasiru qwer1234
[SERVER] OK 200 SID:1001 Login successful. TOKEN:e5f6fe66134afa8974d24ff90b0a39ea7a6e3bff1823f05cefdd6e3da7b1e0ea  (expires after 300 sec inactivity)
[HINT]   To use protected commands, send: TOKEN e5f6fe66134afa8974d24ff90b0a39ea7a6e3bff1823f05cefdd6e3da7b1e0ea
>> TOKEN e5f6fe66134afa8974d24ff90b0a39ea7a6e3bff1823f05cefdd6e3da7b1e0ea
[SERVER] OK 200 SID:1001 Token accepted. Session active.
>> ECHO IT24100139
[SERVER] OK 200 SID:1001 ECHO: IT24100139
>>
```

Figure 4.3: Successful user login and session token generation

This Screenshot shows a successful authentication process via the **LOGIN** command. Upon verifying the user's credentials, the server generates and issues a temporary session token, which is required for executing subsequent protected commands.

4.4 Abuse Protection: Brute-Force Lockout

```
>> LOGIN Rasiru 12345678
[SERVER] ERR 401 SID:1001 Invalid credentials (1/5).
>> LOGIN Rasiru 12345678
[SERVER] ERR 401 SID:1001 Invalid credentials (2/5).
>> LOGIN Rasiru 12345678
[SERVER] ERR 401 SID:1001 Invalid credentials (3/5).
>> LOGIN Rasiru 12345678
[SERVER] ERR 401 SID:1001 Invalid credentials (4/5).
>> LOGIN Rasiru 12345678
[SERVER] ERR 429 SID:1001 Too many failed attempts. Account locked 5 minutes.
>> echo "IT24100139"
```

Figure 4.4: Abuse Protection and Brute-force Lockout

This screenshot proves the implementation of the brute-force protection mechanism. After exceeding the maximum allowed number of failed login attempts, the server actively rejects further requests and locks the account/IP to prevent unauthorized access.

5. Multiprocessing and Concurrency

5.1 Handling Concurrent Client

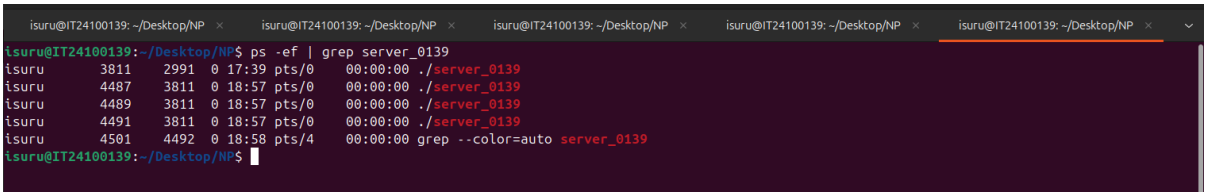

Figure 5.1: Handling Concurrent clients using multiprocessing

This screenshot demonstrates the server's capability to handle multiple concurrent clients. The **PS** command output clearly shows the main parent server process, along with multiple active child processes created via the **fork ()** system call. Each child's process handles a separate client session independently.

- 3811 → Parent Process
- 4487, 4489, 4491 → Child Processes

5.2 Proper Child Cleanup

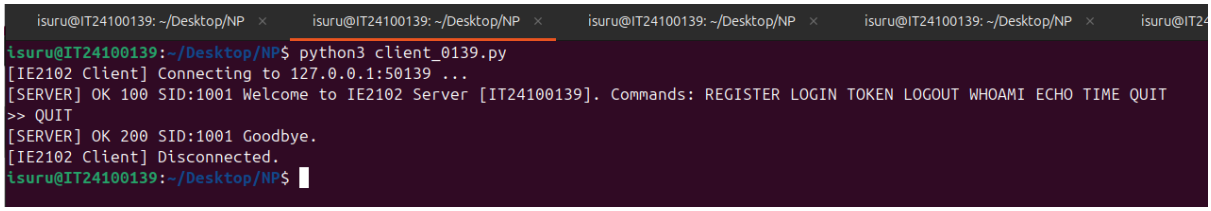

Figure 5.2: Proper Child process cleanup and prevent Zombie process

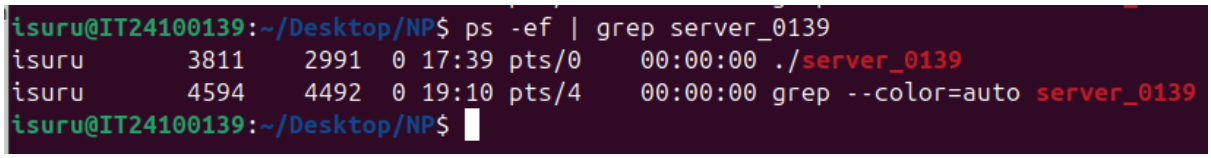

Figure 5.3: Check cleanup

Above Screenshots verifies that the server correctly cleans up terminated child processes. After all clients have disconnected, the subsequent **PS** command confirms that only the parent process remains and there are no Zombie (<defunct>) processes, demonstrating the correct implementation of the **SIGCHLD** handler and **waitpid ().**

6. Persistent Audit Logging

6.1 Server Log file verification

```
isuru@IT24100139:~/Desktop/NP$ cat /srv/ie2102/IT24100139/server_IT24100139.log
[2026-04-09 10:12:07] CLIENT=0.0.0.0:0 PID=4172 USER=- CMD=SERVER_START RESULT=OK
[2026-04-09 10:13:31] CLIENT=127.0.0.1:43536 PID=4198 USER=- CMD=CONNECT RESULT=OK
[2026-04-09 10:14:10] CLIENT=127.0.0.1:43536 PID=4198 USER=isuru CMD=REGISTER RESULT=FAIL-EXISTS
[2026-04-09 10:14:46] CLIENT=127.0.0.1:43536 PID=4198 USER=Rasiru CMD=REGISTER RESULT=OK
[2026-04-09 10:15:12] CLIENT=127.0.0.1:43536 PID=4198 USER=rasiru CMD=LOGIN RESULT=FAIL-NO_USER
[2026-04-09 10:15:48] CLIENT=127.0.0.1:43536 PID=4198 USER=Rasiru CMD=LOGIN RESULT=OK
[2026-04-09 10:16:07] CLIENT=127.0.0.1:43536 PID=4198 USER=Rasiru CMD=TOKEN RESULT=OK
[2026-04-09 10:16:21] CLIENT=127.0.0.1:43536 PID=4198 USER=Rasiru CMD=WHOAMI RESULT=OK
[2026-04-09 10:16:39] CLIENT=127.0.0.1:43536 PID=4198 USER=Rasiru CMD=ECHO RESULT=OK
[2026-04-09 10:16:44] CLIENT=127.0.0.1:43536 PID=4198 USER=Rasiru CMD=TIME RESULT=OK
[2026-04-09 10:16:53] CLIENT=127.0.0.1:43536 PID=4198 USER=Rasiru CMD=LOGOUT RESULT=OK
```

Figure 6.1: Persistent audit logging of server activities

This screenshot shows the content of the **server_IT24100139.log** file, demonstrating the persistent audit logging feature.

Important:

Remember

- *Plagiarism is considered as a serious crime against intellectual property in academic environments.*
- *By including the declaration page mentioned below, you abide by the laws of the institute to accept 0 marks for the report, in case you get caught attempting plagiarism.*

Annexure 1

General Formatting

1. Paper size: A4
2. Contents font & size: Times New Roman, size 12
3. Chapter headings/sub headings font & size: Times New Roman Bold, size 12
4. Page numbering:
   - Cover page should not contain any number
   - Declaration page to table of figures page should be numbered using Roman numbers at bottom center of the page
   - Content pages, references page(s) and annexes (if any) should be numbered using Arabic numbers at the bottom center of the page
5. Margins: 1 inch margins on left, right, top and bottom
6. Line spacing: 1 line spacing

Figure Numbering:

1. Each figure should be numbered at bottom center of the figure along with a figure name.
2. Figure number should start with word 'Figure' followed by the 'main chapter number' and 'figure number within the chapter' separated by a 'period sign (.)' and followed by a 'colon (:)'.

For example,

Figure 2.6: XYZ Connector

Declaration Page:

Declaration page should contain the following. Reports that does not contain this declaration 'AS IS' will not be accepted as valid reports and will be awarded 0 marks.

Declaration

I certify that this report does not incorporate without acknowledgement, any material previously submitted for a degree or diploma in any university, and to the best of my knowledge and belief it does not contain any material previously published or written by another person, except where due reference is made in text